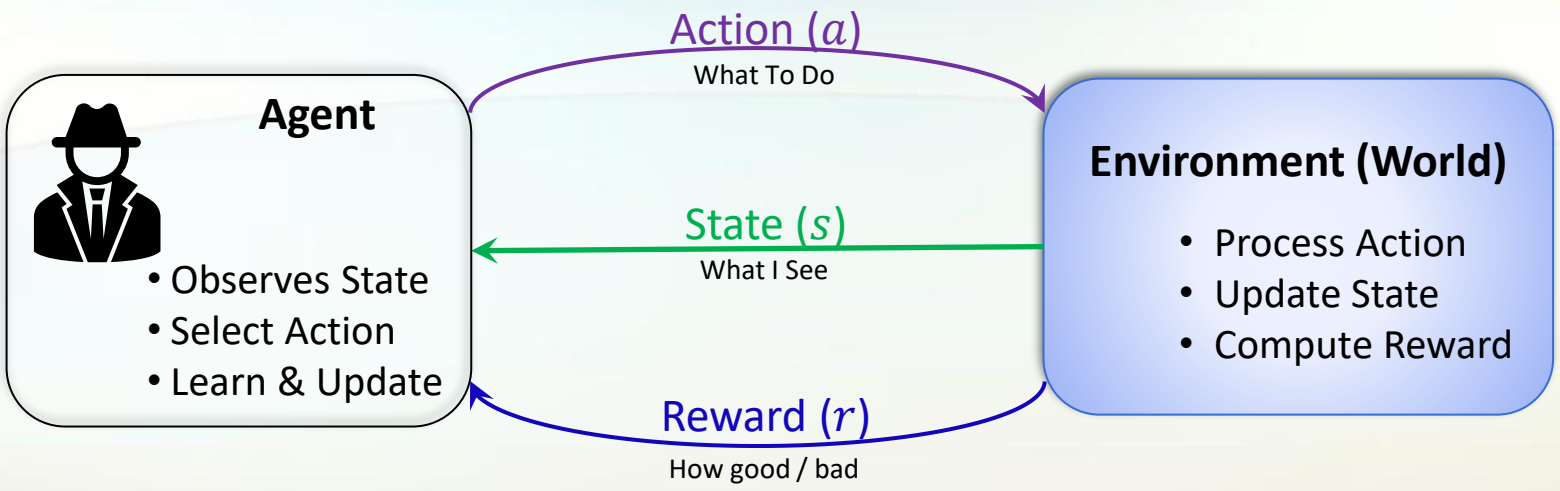
Games and Game Theory

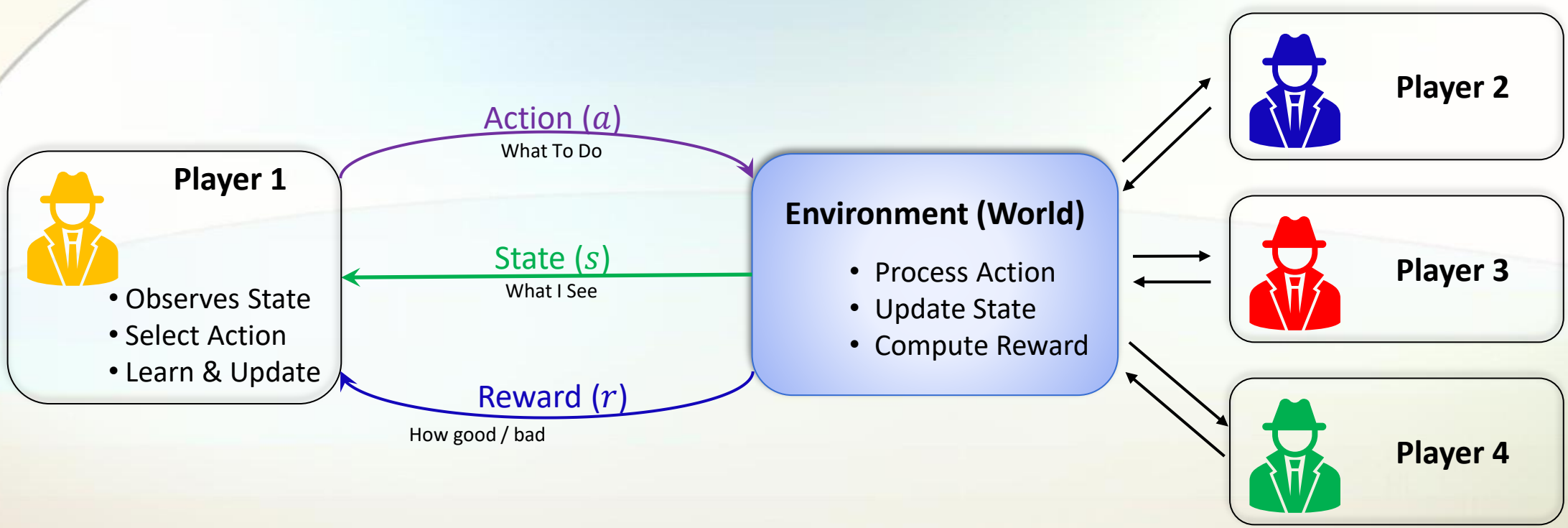
General, Traditional AI Model



Learning Goal: Maximize the total cumulative reward $\sum r_t$

(This is a setup for decision theory, reinforcement learning, planning.)

Game Theory: Multiple Agents



Learning Goal: Maximize the total cumulative reward $\sum r_t$

Now that we have interactions among agents, strategic decision making becomes essential.

Most real-world AI systems operate in **multiagent environments** where success depends on strategic reasoning about other intelligent agents.

Why Study Games in AI?

The future of AI is not just about making smarter individual agents—it is about understanding what happens when multiple intelligent systems interact strategically.

- Bid in an online auction against other AI agents
- Navigate traffic with autonomous vehicles making strategic decisions
- Allocate cloud computing resources among competing users
- Defend against cyber attacks where attackers adapt to your strategy
- Negotiate deals in automated trading systems
- Coordinate drone swarms where each drone must anticipate others' movements
- Design recommendation algorithms that compete for user attention on social platforms
- Manage cryptocurrency mining pools where miners can strategically join/leave
- Price products dynamically against competitors who also use AI pricing
- Allocate bandwidth in 5G networks with competing service requests
- Manage energy grids with multiple producers and consumers making strategic decisions
- Optimize supply chains where suppliers, manufacturers, and retailers have different goals
- Deploy security patrols against adaptive adversaries who learn your patterns

Traditional AI	Game Theory
Agent vs. Static Environment	Agent vs. Intelligent opponents who react to your actions.

The Challenge: Other agents are intelligent, adaptive, and have their own goals!

This changes everything - optimal strategies must account for strategic interaction.

From Optimization to Strategy

Optimal strategies must account for strategic interaction.

	Traditional AI	Game Theory
Solution Concept	Find the objectively “best” action	Find the best action given what others will do
Optimization Target	Maximize your own utility	Maximize your utility while others are trying to maximize theirs
Predictability	Environment is static or follows known patterns	Environment adapts and responds to your strategy
Strategy Complexity	“If I do X, outcome Y will happen”	"If I do X, others will do Z, so outcome Y might happen"
Equilibrium vs. Optimality	Seek optimal solutions	Seek stable solutions (Nash equilibrium) that may be suboptimal

This is the fundamental shift. We cannot optimize in isolation anymore. We must reason about the strategic interactions and feedback loops created by multiple intelligent decision-makers.

Modeling Games: Properties

Just as we classify search problems by their properties (observable, deterministic, episodic), we must characterize games by their strategic structure.

Game Property	Category 1	Category 2	Example
Number of players	Single player	Multiple players	Chess (2), Poker (multiple)
Action space	Finite	Infinite	Rock-Paper-Scissors vs. Tennis
Outcome	Deterministic	Random/Stochastic	Chess vs. Poker (card dealing)
Payoff structure	Zero-sum	General-sum	Chess vs. Prisoner's Dilemma
Move timing	Sequential	Simultaneous	Chess vs. Rock-Paper-Scissors
Information visibility	Perfect information	Imperfect information	Chess vs. Poker (hidden cards)
Knowledge of rules	Complete information	Incomplete information	Chess vs. Sealed-bid auction

Properties I

1. Number of Players

- 1 or more players; however, we are usually interested in ≥ 2 players
- Typically, a finite number of players.

2. Action Space: set of possible actions an agent can choose from.

Finite:

- Tic-tac-toe (9 squares),
- Checkers (piece movements)
- Chess (finite, but large: $\sim 10^{120}$.

Infinite:

- Price setting ($0.01, $0.02, $0.03...),
- Robot navigation (continuous coordinates)

Finite action spaces allow exhaustive analysis; infinite spaces require approximation techniques.

Properties II

3. Deterministic or Random: Is there chance in the game?

Deterministic:

- Chess
- Checkers
- Go
- Connect Four
- Azul
- Splendor

Non-Deterministic:

- Scrabble
- Poker
- Backgammon
- Bridge
- Machi Koro
- Settlers of Catan

4. Sum of Payoffs

Zero-Sum Games	General-Sum Games
⚔️ One player's win = other's loss	🤝 Players can both win or both lose
Pure competition	Mixed cooperation/competition
Examples: Rock-paper-scissors, Chess	**Examples:** Prisoner's dilemma, Traffic coordination
Easier to solve using minimax algorithm	Requires Nash equilibrium concepts

Example: Prisoner's Dilemma

Two suspects are arrested and held in separate cells. They cannot communicate with each other.

The prosecutor offers each prisoner a deal:

- **If one betrays and one stays silent**: Betrayer goes free, Silent gets 3 years
- **If both stay silent**: Both get 1 year
- **If both betray**: Both get 2 years

	Stay Silent	Betray
Stay Silent	$(-1,-1)$	$(-3,0)$
Betray	$(0,-3)$	$(-2,-2)$

Payoffs (years in prison)

Rational choice for each prisoner: Betray

Best outcome for both together: Stay Silent

This shows how ***individual rationality can lead to collectively irrational outcomes***. Each prisoner, acting in their own self-interest, creates a worse result for everyone.

Real-World Examples:

- Price wars between companies
- Traffic congestion (everyone takes "faster" route)

Properties III

5. Sequential or Simultaneous Moves?

All players take action at the same time: Simultaneous

Players take turns: Sequential

(Payoff is often only revealed at end of game.)

Sequential:

- Chess
- Checkers
- Go
- Connect Four
- Azul
- Splendor

Simultaneous:

- Rock-Paper-Scissors
- Red Light, Green Light
- Musical Chairs
- Tennis Serves
- Real-time strategy games
- Wordle

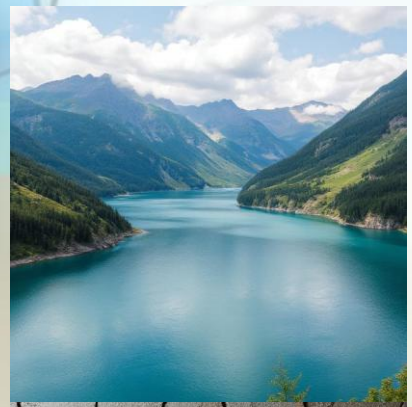
Property	Sequential Games	Simultaneous Games
Timing	Players take turns, one after another	All players act at the same time
Information	Can observe opponents' previous moves	Must choose without seeing others' choices
Strategy	React and adapt to what others have done	Must anticipate what others will do
Representation	Game trees, extensive form	Payoff matrices, normal form
Solution Method	Backward induction, minimax	Nash equilibrium, dominance
Commitment	Can change strategy based on observations	Must commit before seeing others' actions
AI Algorithms	Game trees and Minimax-style algorithms	Normal form representation and equilibrium concepts

Properties IV

6. Perfect vs. Imperfect Information

Each player knows the complete game state at all times: Perfect

Players have hidden or private information: Imperfect



Perfect:

- Chess
- Checkers
- Go
- Tic-Tac-Toe
- Connect Four

Imperfect:

- Poker
- Battleship
- Bridge
- Magic: The Gathering
- Clue / Cluedo

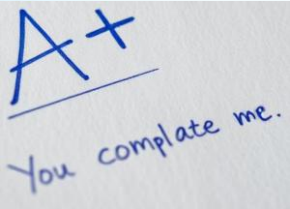
Perfect Information	Imperfect Information
All information visible	Some information hidden
Pure strategy/logic	Strategy + psychology
Deterministic optimal play	Probabilistic reasoning
No bluffing possible	Bluffing and deception
Complete game tree analysis	Belief-based decision making
Minimax, Game tree search work well (Deep Blue, AlphaGo)	**Need probabilistic models, opponent modeling, game theory (poker bots)**

Properties IV

7. Complete vs. Incomplete Information

All players know the rules, payoffs, and possible strategies of all players — Complete

Players do not know other players' payoffs, types, or capabilities — Incomplete



Complete:

- Chess
- Poker
- Rock-Paper-Scissor
- Prisoner's Dilemma

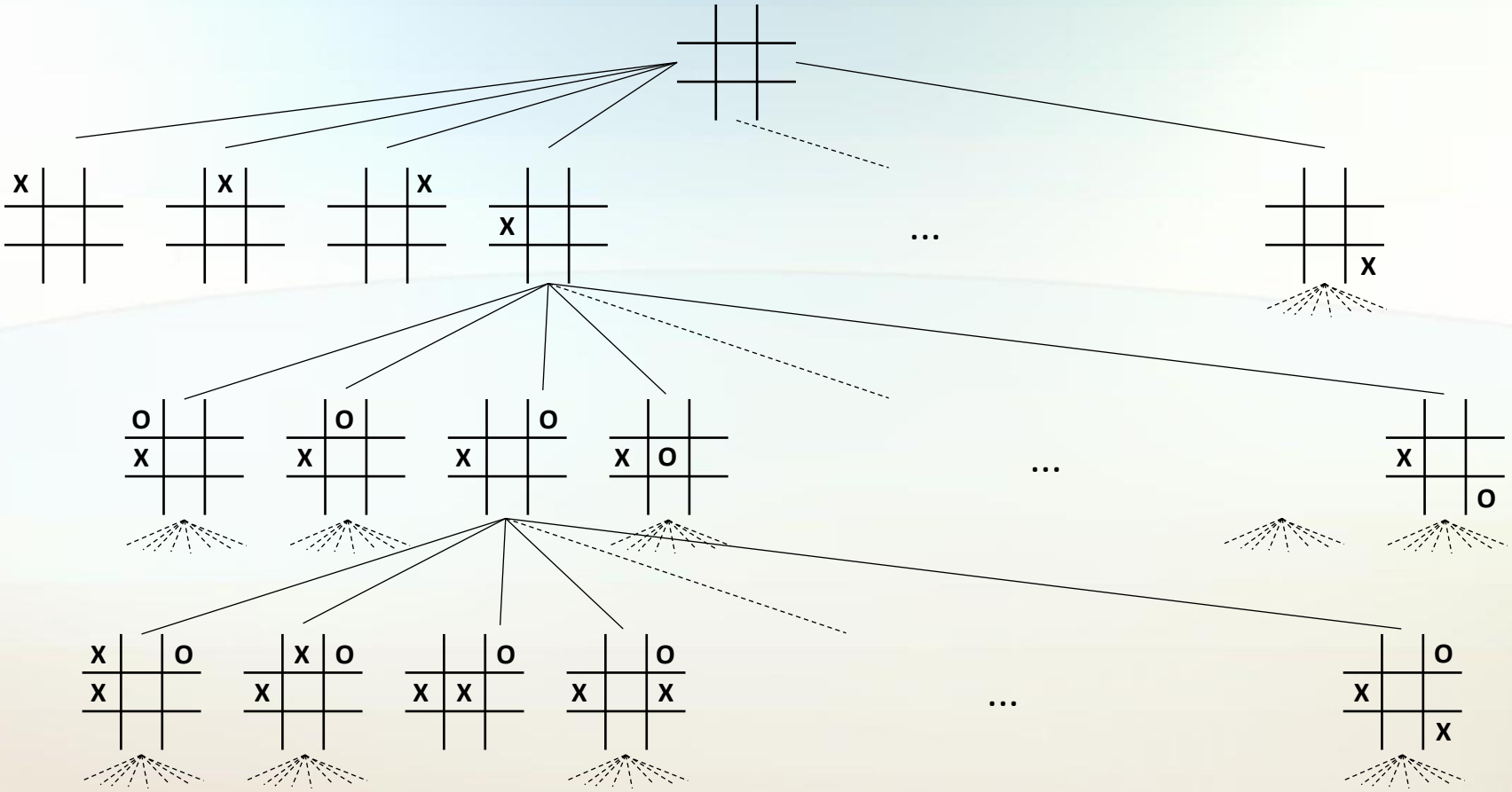
Incomplete:

- Mafia (game)
- Job interviews
- Sealed-bid auctions
- Dating Apps

Complete Information	Incomplete Information
Know all players' payoffs	Don't know opponents' payoffs
Understand all strategies	Uncertain about opponents' capabilities
Fixed, known game rules	May not know opponents' constraints
Can predict rational responses	Must guess opponents' motivations
Standard game theory applies	Need Bayesian game theory
Nash equilibrium, dominance analysis work directly	**Need mechanism design, signaling theory, auction theory**

Sequential-Move Games

Game Tree Example



Nodes: Game states
Edges: Available moves
Leaves: Terminal states with payoffs
Strategy: Complete plan for all possible situations

~ 255168 possible games

2-Nim

Game Rules

Setup: Two piles of sticks, each with 2 sticks. **Initial State:** (| | , | |)

Move: Each player takes one or more sticks from *one* pile.

Losing condition: The player who takes the last stick loses.

Players: **Max** (maximizing) and **Min** (minimizing)

Scoring: **Max** wins → +1, **Max** loses → -1

This is a zero-sum game as **Min**-*score* := −**Max**-*score*.

Sample Playthrough

| | , | |

Max takes 1

| , | |

Min takes 2

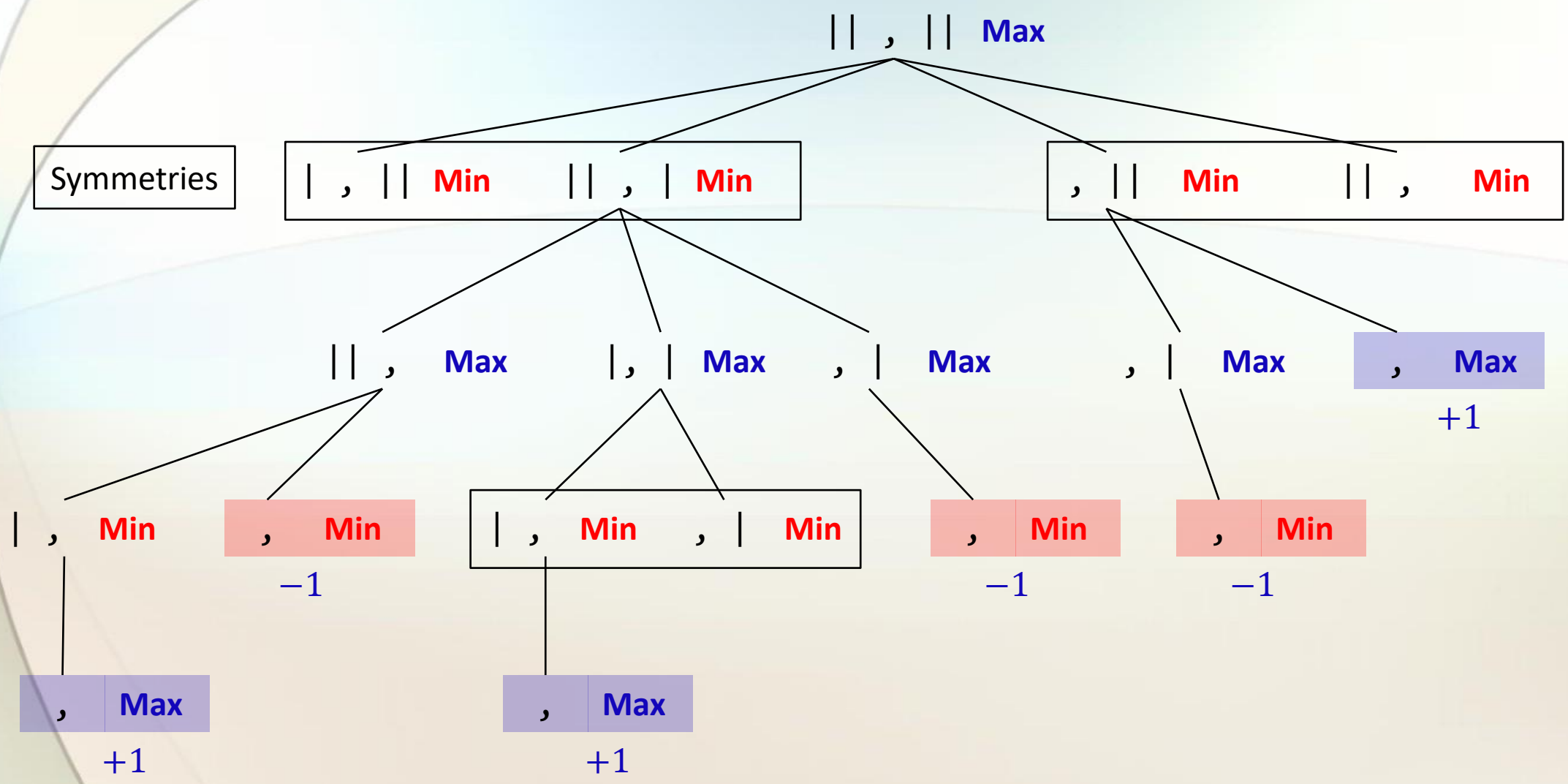
| ,

Max takes last

,

Max-*score* := -1

2-Nim Game Tree



Max wants the largest score.
Min ‘wants’ the smallest score.

Minimax Algorithm

```
function MINIMAX-DECISION(state) returns an action
  return arg max_{a ∈ ACTIONS(s)} MIN-VALUE(RESULT(state, a))
```

```
function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s, a)))
  return v
```

Payoff Function

Compare current best with this move's outcome

```
function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s, a)))
  return v
```

Track **Min**'s best choice (lowest score)

Figure 5.3 An algorithm for calculating minimax decisions. It returns the action corresponding to the best possible move, that is, the move that leads to the outcome with the best utility, under the assumption that the opponent plays to minimize utility. The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state. The notation $\text{argmax}_{a \in S} f(a)$ computes the element a of set S that has the maximum value of $f(a)$.

Complexity

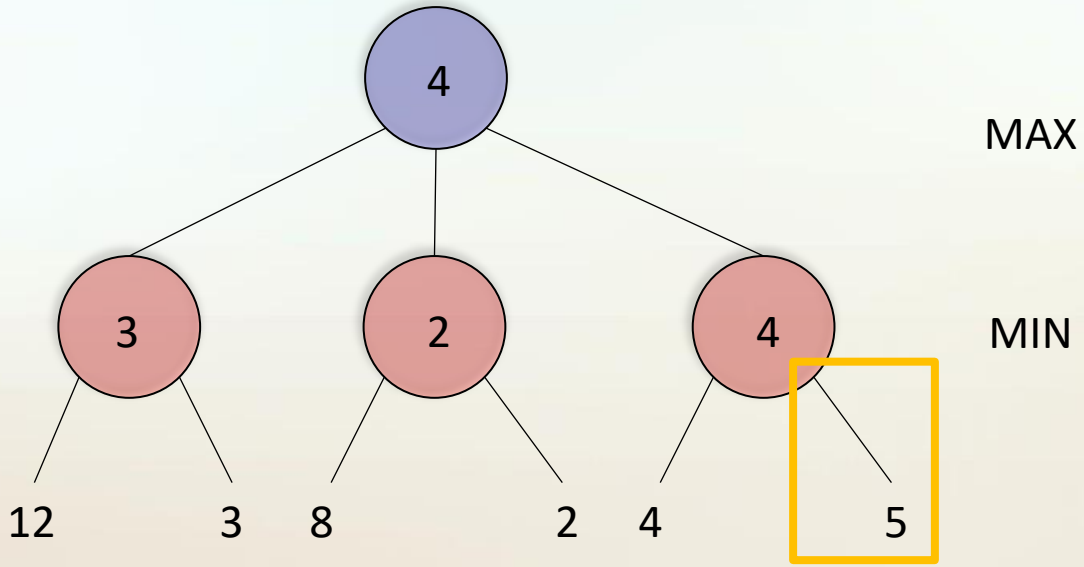
Let b be branching factor and m be the maximum depth.

Time: $O(b^m)$

Space: $O(bm)$ (depth-first)

Simple Example

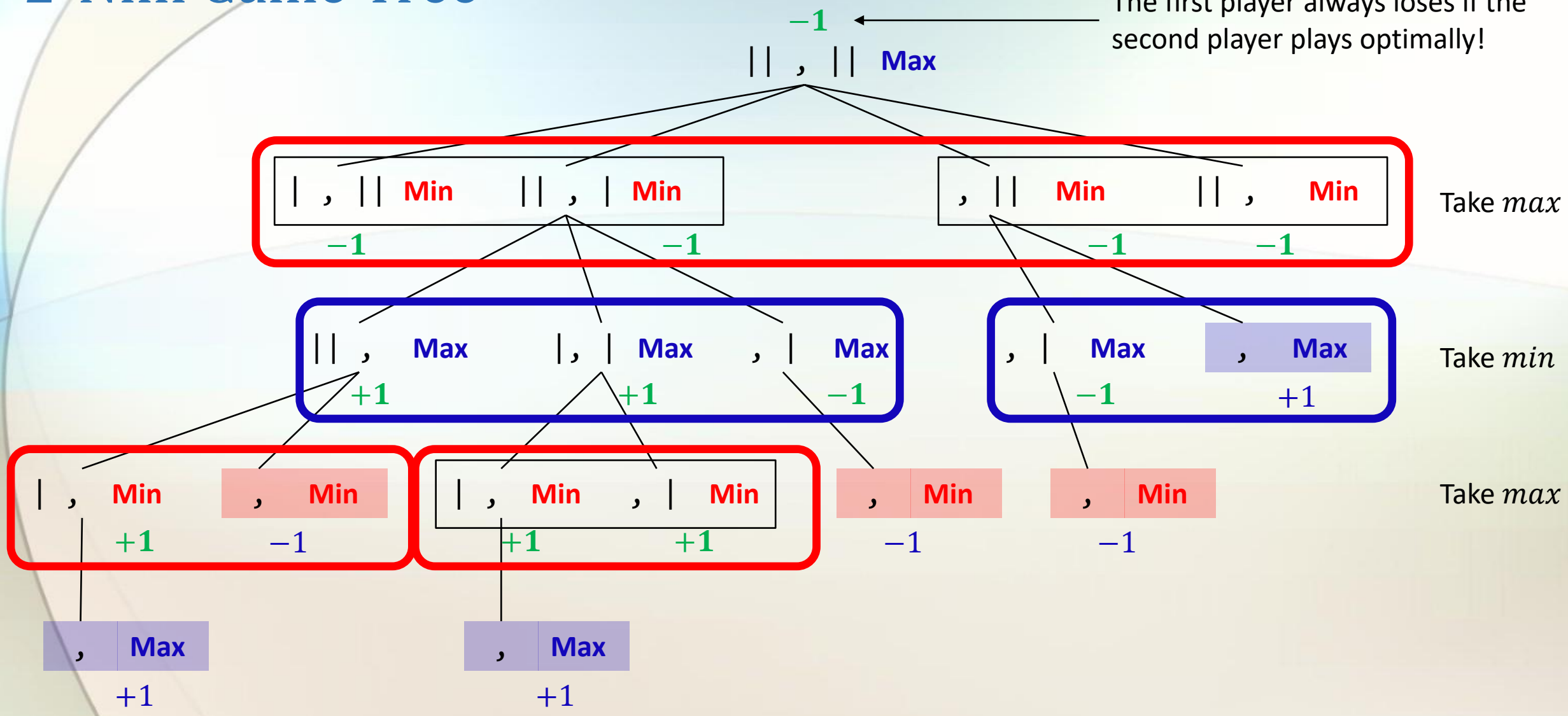
Visit is via DFS and values propagate bottom-up:



Do we need to evaluate this node?

Result: Optimal play leads to value of 4.

2-Nim Game Tree



Max wants the largest score: on **Max**'s turn, take max of children.
Min 'wants' the smallest score: on **Min**'s turn, take min of children.

Questions

We are playing a game where Player A goes first and has 4 possible moves.
Player B goes next and has 3 possible moves.
Player A goes next and has 2 possible moves.
Player B then has one possible move.
How many nodes are there in the minimax tree, including termination nodes?

Solution.

$$\boxed{1} + 4 + 4 \cdot 3 + 4 \cdot 3 \cdot 2 + \boxed{4 \cdot 3 \cdot 2} = 65$$

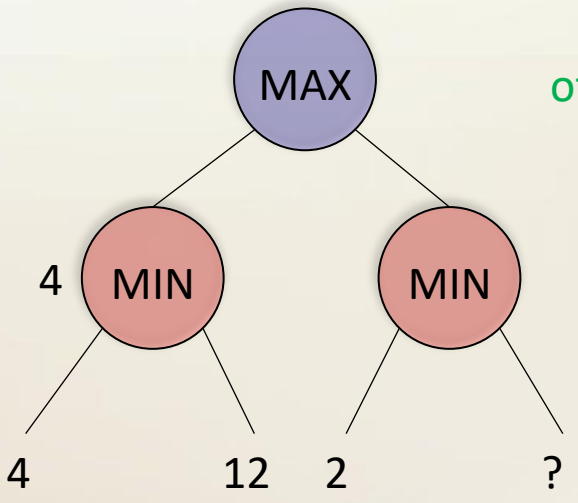
root

leaves

During minimax tree search, must we examine every node?

No.

Consider layer k, where we take the max of all the mins of its children at layer $k+1$.
If the current value of a min node at $k+1$ already smaller than the current max, we do not need to continue the minimization.



other	other	other	other
1	2	2	2
1	3	5	13

Can We Do Better Than Strict Minimax?

Minimax Complexity

Time: $O(b^m)$

Space: $O(bm)$
(depth-first)

For complex games, this is computationally prohibitive:

Game	Branching Factor	Average Game Length	Total Positions
Chess	≈ 35	≈ 40 moves	$\approx 10^{50}$
Go	≈ 250	≈ 150 moves	$\approx 10^{170}$
Checkers	≈ 10	≈ 70 moves	$\approx 10^{20}$

Downside: we examined the entire tree. An idea to speed things up: pruning

- If we can identify "bad" branches early, we can skip exploring them
- Goal: want the same minimax value, but faster

Alpha-beta pruning can reduce the effective branching factor significantly:

Best case: $O\left(b^{m/2}\right)$, a square root improvement.

Real example: Deep Blue reduced branching from $35 - 40$ to ~ 6.

Alpha-Beta Pruning

```
function ALPHA-BETA-SEARCH(state) returns an action
  v ← MAX-VALUE(state, −∞, +∞)
  return the action in ACTIONS(state) with value v

function MAX-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if v ≥ β then return v
    α ← MAX(α, v)
  return v

function MIN-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← +∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s,a) , α, β))
    if v ≤ α then return v
    β ← MIN(β, v)
  return v
```

Figure 5.7 The alpha–beta search algorithm. Notice that these routines are the same as the MINIMAX functions in Figure 5.3, except for the two lines in each of MIN-VALUE and MAX-VALUE that maintain α and β (and the bookkeeping to pass these parameters along).

Pass around α and β.

Update α at MAX nodes
Update β at MIN nodes

α: MAX's best option so far
β: MIN's best option so far

Cutoff Decisions

If value $\geq \beta$, prune remaining children.

If value $\leq \alpha$, prune remaining children.

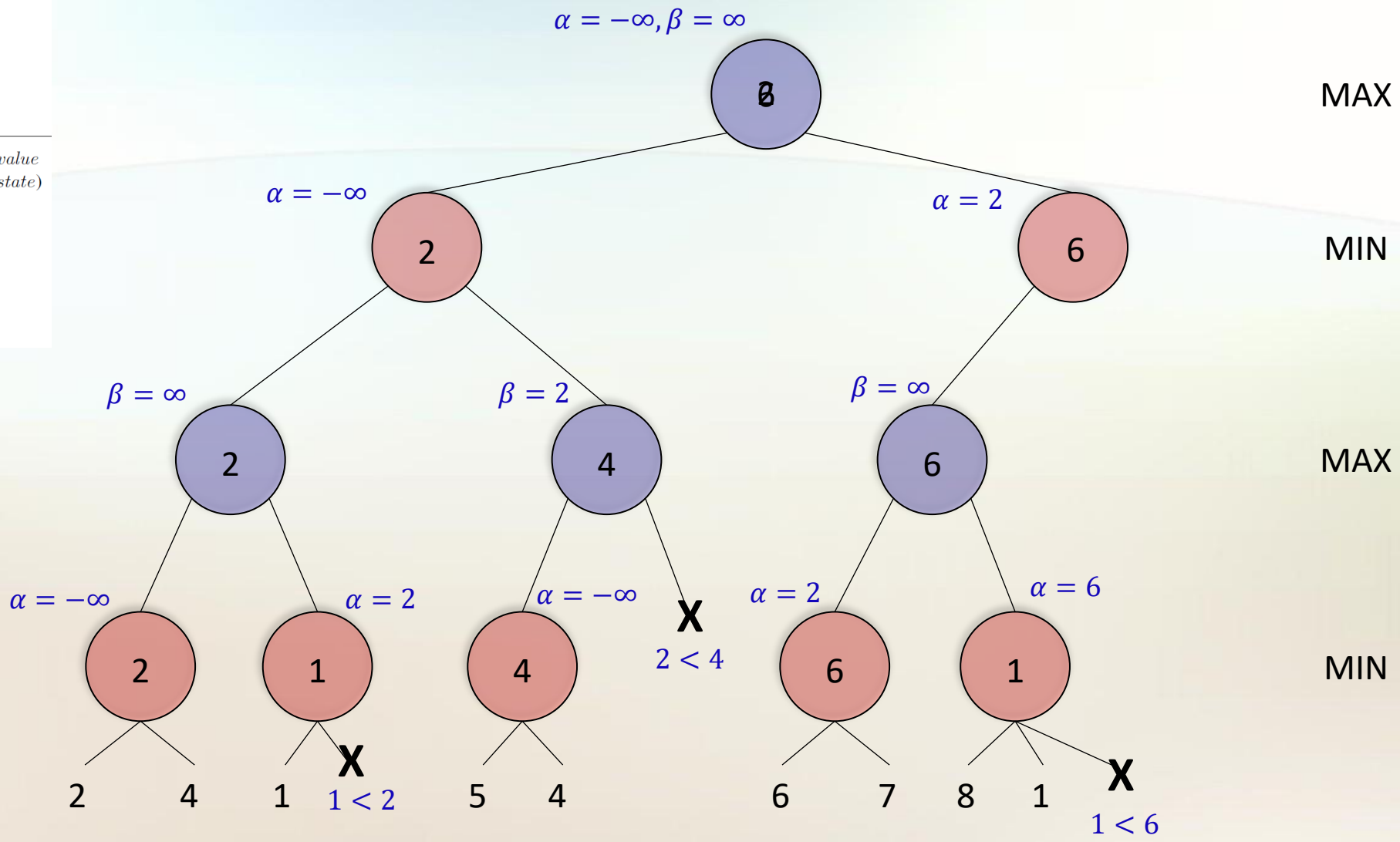
```
function ALPHA-BETA-SEARCH(state) returns an action
  v ← MAX-VALUE(state, −∞, +∞)
  return the action in ACTIONS(state) with value v
```

```
function MAX-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if v ≥ β then return v
    α ← MAX(α, v)
  return v
```

```
function MIN-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← +∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s,a) , α, β))
    if v ≤ α then return v
    β ← MIN(β, v)
  return v
```

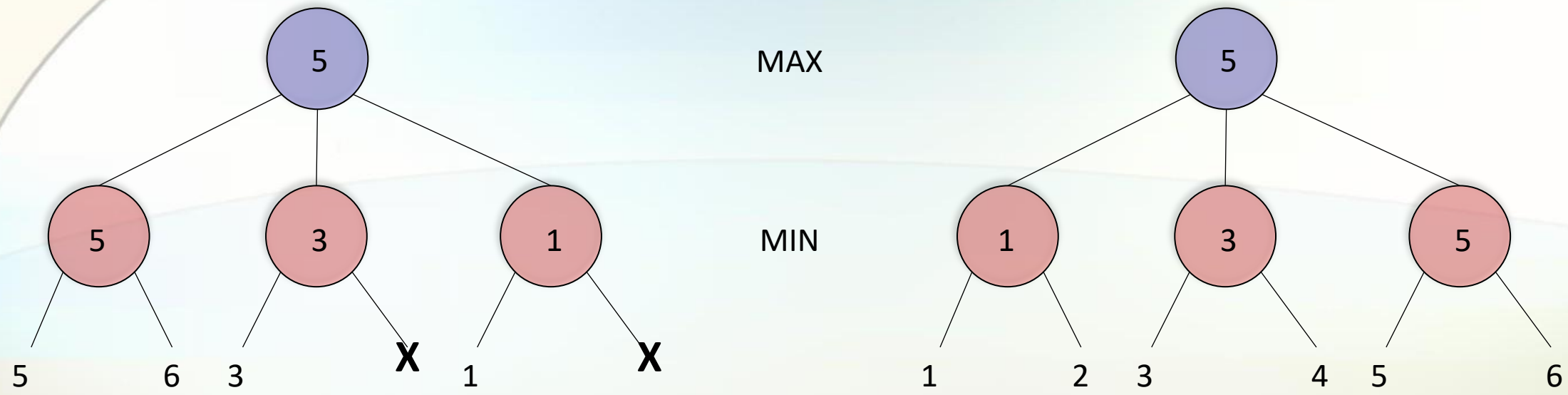
Example

MIN prunes if $v \leq \alpha$
MAX prunes if $v \geq \beta$



Compute the minimax in the following tree. Identify any branches that can be pruned in the computation.

Alpha-Beta Effectiveness: Order Matters



Move Ordering Matters!

- Best moves first → Maximum pruning opportunities
- Use domain heuristics to prioritize promising moves
- Iterative deepening improves move ordering over time: Save / use best moves from shallow searches to order moves at deeper depths.
- Killer moves and history tables guide search

Minimax with Heuristics

The Challenge: Even with alpha-beta pruning, complete game trees are often too computationally large:

Chess: $\sim 10^{50}$ possible games

Go: $\sim 10^{170}$ possible games.

We need to limit the search depth to a depth d.

What to do at depth d, but have not yet terminated?

Use a heuristic evaluation function $e(x)$

```
function MINIMAX(x, d) returns an estimate of x's utility value
  inputs: x, current state in game
          d, an upper bound on the search depth
  if x is a terminal state then return Max's payoff at x
  else if d = 0 then return e(x)
  else if it is Max's move at x then
      return max{MINIMAX(y, d−1) : y is a child of x}
  else return min{MINIMAX(y, d−1) : y is a child of x}
```

Heuristic Evaluation Functions

General Principles for $e(x)$:

- Must be fast to compute
- Should correlate with winning probability
- Often linear combination of features
- Domain-specific knowledge crucial

$e(x)$ can be any computable function of x:

$$e(x) = w_1 f_1(x) + w_2 f_2(x) + \cdots + w_n f_n(x)$$

Feature Engineering: The art of choosing good features that capture important aspects of game positions.

Modern approaches use machine learning to learn evaluation functions automatically! Instead of manually designing features, modern chess engines (like AlphaZero) use machine learning to:

- Automatically learn which features matter most.
- Build their own evaluation functions by training on millions of positions and outcomes.
- Use neural networks to capture complex patterns that humans might miss.

Heuristic Evaluation Function Example

For Chess, we might evaluate the current state of the board and an overall player's position in the game using:

$$
\begin{aligned}
e(game_state) = {} & 9 \cdot (Q_{white} - Q_{black}) + \\
& 5 \cdot (R_{white} - R_{black}) + \\
& 3 \cdot (B_{white} - B_{black}) + \\
& 3 \cdot (K_{white} - K_{black}) + \\
& 1 \cdot (P_{white} - P_{black}) + mobility + king_safety + \cdots
\end{aligned}
$$

Material Balance: Calculates the difference in piece count between White and Black.

Mobility: Rewards positions with more legal moves (greater flexibility).

King Safety: Penalizes exposed or vulnerable kings.

Interpretation:

- Positive score → Advantage for White
- Negative score → Advantage for Black
- Zero → Balanced board state

There are other factors (represented by ⋯):

- Control of the center
- Pawn structure (e.g., doubled, isolated)
- Piece activity (e.g., rooks on open files)
- Tactical threats (e.g., forks, pins)

Beyond Minimax: Monte Carlo Tree Search (MCTS)

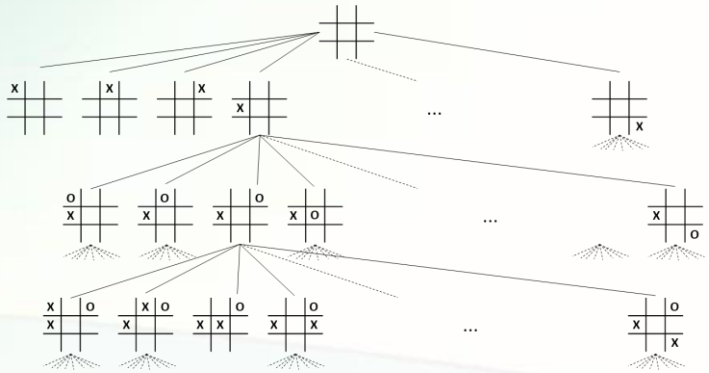
Traditional Alpha-Beta Limitations

- **Designing evaluation functions is challenging** – It's difficult to define what makes a Go position strong or advantageous.
- **Massive branching factor** – Each turn in Go offers around 250 possible legal moves, making decision trees extremely complex.
- **Strategic depth exceeds tactical calculation** – Purely tactical analysis often fails to capture the broader strategic elements of the game.
- **Expert knowledge is essential for effective heuristics** – Crafting good evaluation rules requires deep domain expertise.

MCTS

- **Bypass traditional evaluation functions** – Rely on simulated gameplay to assess positions instead.
- **Learn through experience** – Develop intuition by playing numerous randomized games.
- **Navigate the exploration–exploitation trade-off** – Balance trying new moves with focusing on those that show promise.
- **Minimal domain knowledge required** – The approach works effectively even without deep subject expertise.

The Idea Behind Monte Carlo Tree Search



Idea

Learn which moves are good by playing out random games.

Process

- ***Selection***: Select a promising *path* through the game tree
- ***Expansion***: Expand the tree by adding a new node
- ***Rollout***: Simulate a random game from that position
- ***Backpropagation***: Backpropagate the result up the tree

We will repeat these operations ('in sequence'); usually time is the limiting factor for the search.

Result

Nodes visited more often in winning simulations get higher priorities.

MCTS Selection Operation

We will navigate from the root to a *leaf* using the ***selection*** policy based on *Upper Confidence Bound 1*:

$$UCB1(n) = \boxed{\frac{wins_n}{visits_n}} + C \cdot \boxed{\sqrt{\frac{\ln(parent_visits_n)}{visits_n}}}$$

Exploitation Exploration

where:

- n: a node (state) in the search tree
- $wins_n$: number of wins when passing through node n
- $visits_n$: number of times n has been visited
- $parent_visits_n$: number of times the parent of n has been visited
- C: a (tunable) positive constant

* To avoid division by 0, most implementations have $UCB1 = \infty$ for any unvisited node ensuring that every node is visited at least once, promoting exploration.

MCTS Selection Example

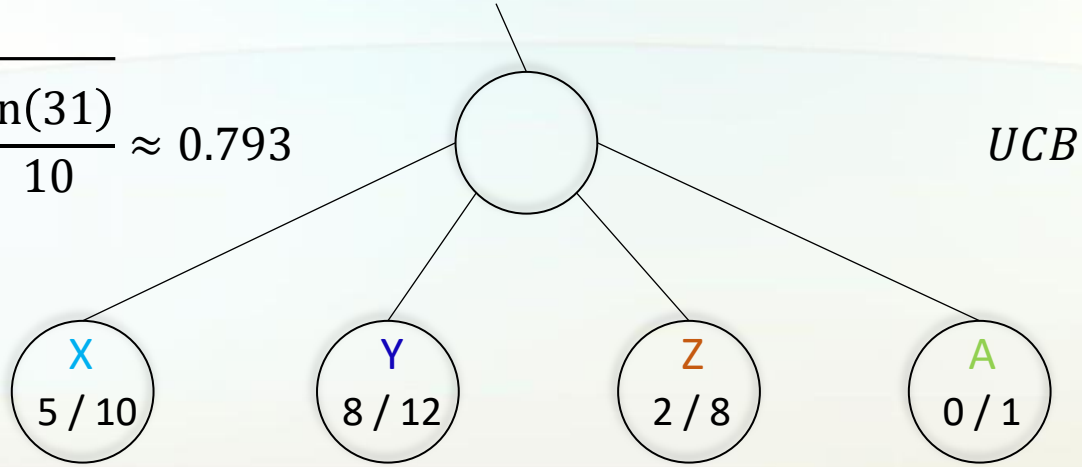
$$UCB1(n) = \frac{wins_n}{visits_n} + C \cdot \sqrt{\frac{\ln(parent_visits_n)}{visits_n}}$$

Let $C = 0.5$

$$UCB1(X) = \frac{5}{10} + 0.5 \cdot \sqrt{\frac{\ln(31)}{10}} \approx 0.793$$

$$UCB1(A) = \frac{0}{1} + 0.5 \cdot \sqrt{\frac{\ln(31)}{1}} \approx 0.927$$

After several iterations…



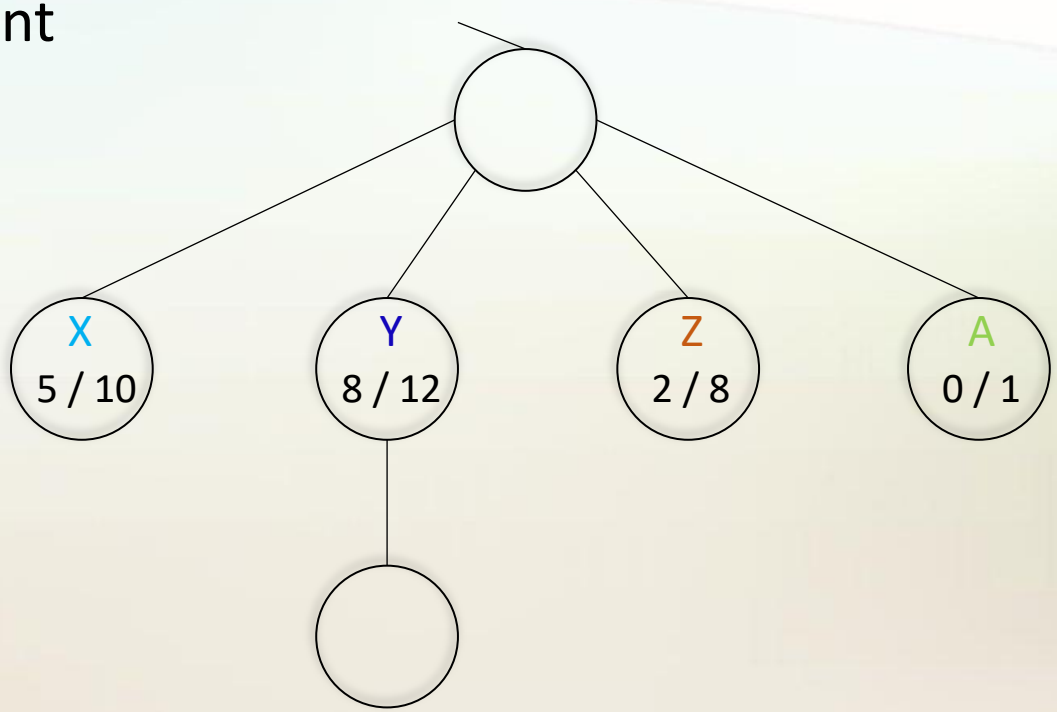
$$UCB1(Y) \approx 0.934 \quad UCB1(Z) \approx 0.578$$

$0 < C < 1$	$C = 1$	$C > 1$
Exploitation	Equal	Exploration

MCTS Expansion

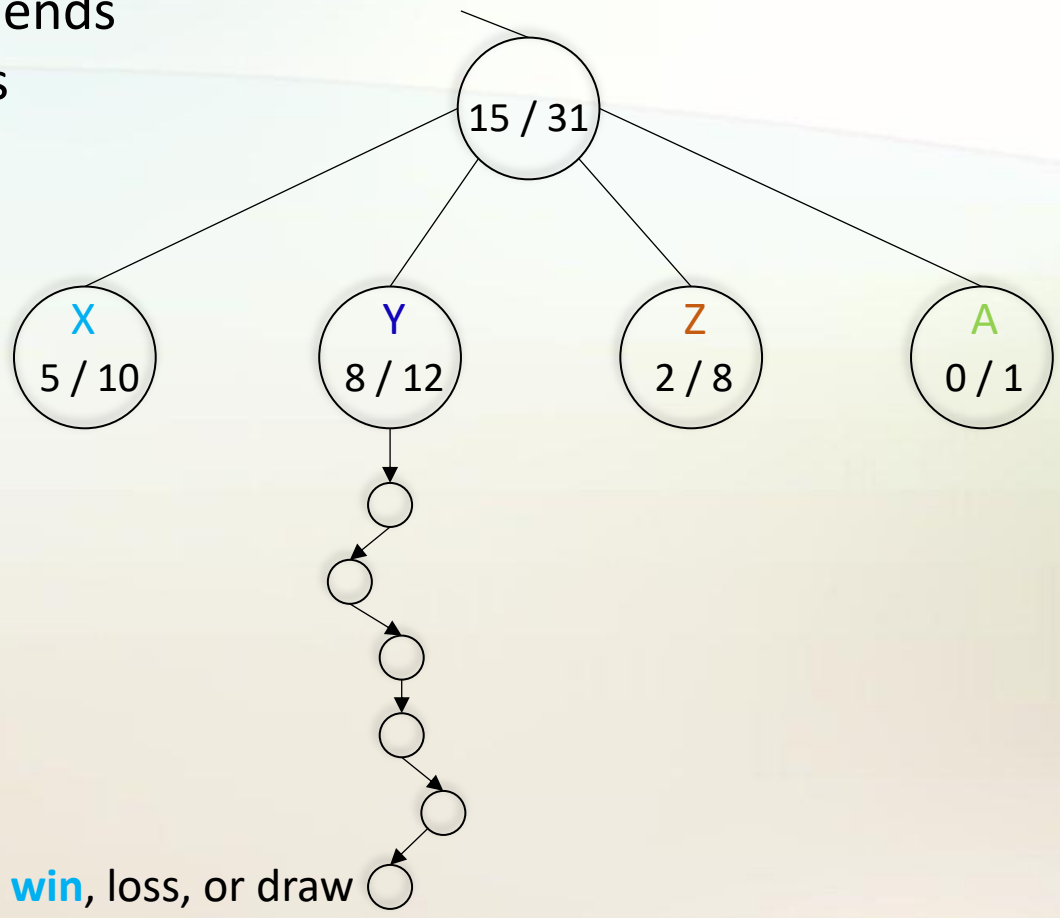
Once a leaf is encountered, we ***expand***:

- Add one new child to the selected leaf node
- Choose an untried move from the current position



MCTS Simulation (Rollout)

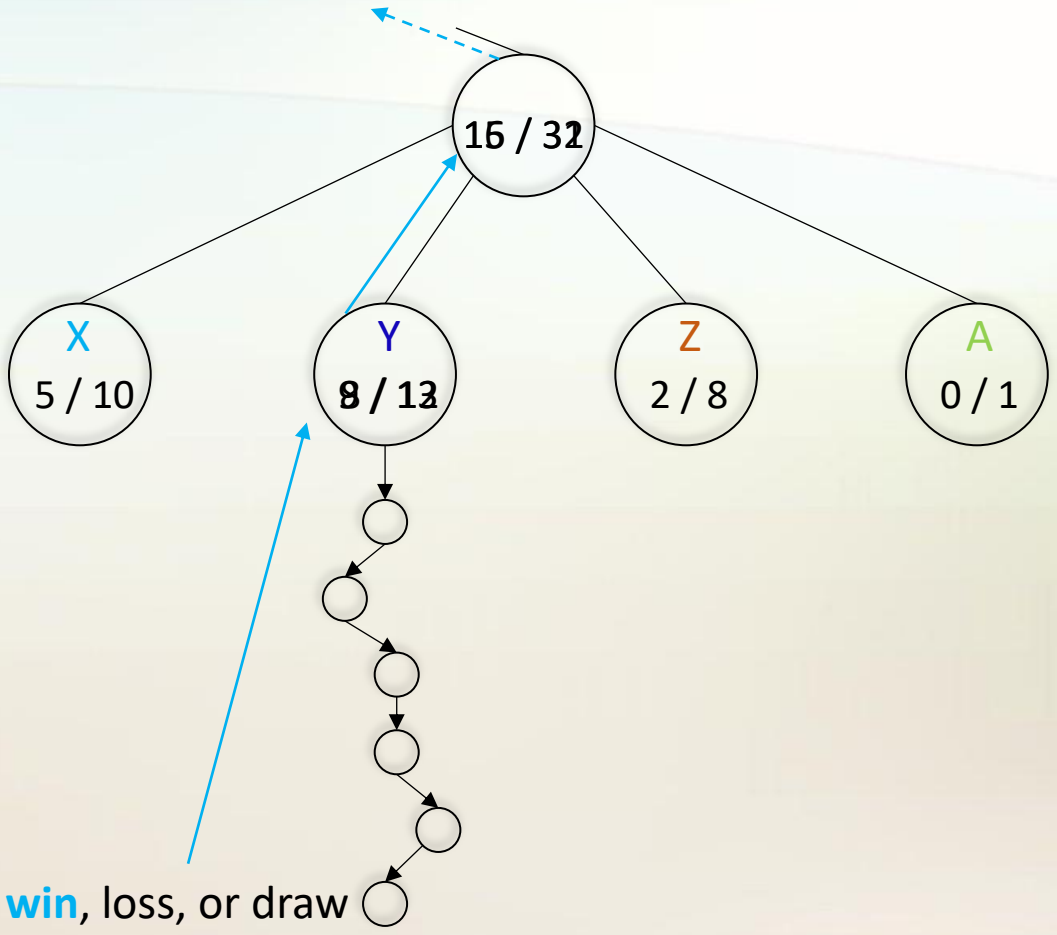
The ***Rollout*** operation simulates the rest of the game from the new node using random moves until it ends in a win, loss, or draw — the randomness here is what gives MCTS its Monte Carlo character.



MCTS Backpropagation

When a result is achieved from Rollout, we ***backpropagate*** the result by:

- Update all nodes on the path from root to new leaf
- Increment visit count for each node
- Update win totals based on simulation result



The process then repeats, starting with the selection of a new path to a leaf node—guided by $UCB1$ at each step along the way.

MCTS, In Summary

The key: MCTS does not need a heuristic evaluation function—just simulation statistics!

Advantages	Disadvantages
No evaluation function needed	Requires many simulations
Self-improving over time	May miss precise combinations that dramatically change the game
Handles high branching factors	Slower convergence
Easy to parallelize	Memory intensive
Domain-independent	Statistical (not exact)

	Alpha-Beta	MCTS
Evaluation	Hand-crafted function	Random simulations
Search	Deterministic, exhaustive	Probabilistic, selective
Domain Knowledge	High requirement	Minimal requirement
Branching Factor	Struggles with high branching factor	Handles high branching factor well
Parallelization	Difficult	Natural
Convergence	Exact (given time)	Asymptotic

MCTS is like “adaptive iterative deepening”—it automatically decides where to spend computational effort based on what looks most promising, rather than systematically deepening everywhere.

Simultaneous-Move Games: Normal Form Games

From Game Trees to Payoff Matrices

We have analyzed sequential games using game trees and minimax. But what about ***simultaneous*** games where players act at the same time?

Problem We do not have a tree structure: players cannot observe and react to each other's moves.

Solution Represent the game as a **payoff matrix** listing all strategy combinations and their outcomes.

Sequential Games	Simultaneous Games
Game tree	Payoff matrix
Players alternate	Players act simultaneously
Minimax / backward induction	Equilibrium analysis
Can observe opponent's moves	Must anticipate opponent's choice

Any sequential game can also be converted to matrix form by enumerating all complete strategies—connecting these two representations.

Normal Form Game

We have seen that strategic interactions are everywhere. But how do we **analyze** these complex situations systematically?

Our goal is a mathematical description of ***simultaneous*** games (e.g., Prisoner's Dilemma).

A ***normal form game*** consists of:

	Definition	**Example: Prisoner's Dilemma**
Players (or Agents)	Decision-makers: $N = \{1, 2, \dots, n\}$	Two prisoners
Action Sets	For all $p \in N$, A_p defines a set of actions.	$A_1 = \{Stay\ Silent, Betray\}$ $A_2 = \{Stay\ Silent, Betray\}$
Strategy Profile	A combination of one action from each player $S = (a_1, a_2, \dots a_n)$ where $a_p \in A_p$.	What both prisoners actually do; for example, $(Betray, Betray)$.
Payoff Functions	Reward for player p depends on everyone's choices: $u_p(S)$.	Years in prison (negative numbers because prison is bad); e.g., $u_1(Stay\ Silent, Betray) = -3$

Normal Form Game Representation

The game is typically displayed as a matrix (for two-player games) or higher-dimensional table (for games with more players). In a two-player game:

In a two-player game:

- Rows represent Player 1's (P_1) strategies
- Columns represent Player 2's (P_2) strategies
- Cell contains the payoffs for both players, written: (P_1 payoff, P_2 payoff)

Prisoner's Dilemma

	Stay Silent	Betray
Stay Silent	$(-1,-1)$	$(-3,0)$
Betray	$(0,-3)$	$(-2,-2)$

Assumption for Normal Form Games: Players choose strategies simultaneously, without knowing what the other players will choose; players must commit to their decisions before observing others' actions.

Example

Pepsi and ***Coca-Cola*** must decide whether to launch major advertising campaigns. Both companies compete for the same soft drink customers worldwide.

Each company has two choices:

Advertise: TV commercials, celebrity endorsements, and marketing campaigns

Don't Advertise: Skip the campaigns and save advertising costs

The Strategies:

- When ***Coke*** advertises and ***Pepsi*** does not, ***Coke*** can gain market share.
- When both advertise, they spend money but maintain their relative positions.
- When neither advertises, both save money but keep existing market shares.

Coke vs. ***Pepsi***: Payoff Matrix

(in billions)	***Coke*: Advertise**	***Coke*: Don't**
***Pepsi*: Advertise**	$(1,1)$	$(8,2)$
***Pepsi*: Don't**	$(2,8)$	$(6,6)$

Strong temptation to advertise: potential gain is enormous.

If both companies think this way, they end up in the worst outcome for both.

Dominant Strategies

A ***dominant strategy*** is one that yields the highest payoff regardless of what other players do.

Definition. Let G be a normal form game with players $N = \{1, 2, \dots, n\}$, S_i as the strategy set for player i and a utility function for player i defined as

$$u_i: S_1 \times S_2 \times \cdots \times S_n \to \mathbb{R}.$$

For player i, strategy $s_i \in S_i$ with $s_i \neq s_i{}'$ is ***strictly dominant*** if and only if:

$$u_i(s_i, s_{-i}) > u_i(s_i{}', s_{-i})$$

for all $s_i{}' \in S_i \setminus \{s_i\}$ and for all $s_{-i} \in S_{-i}$.

Notation:

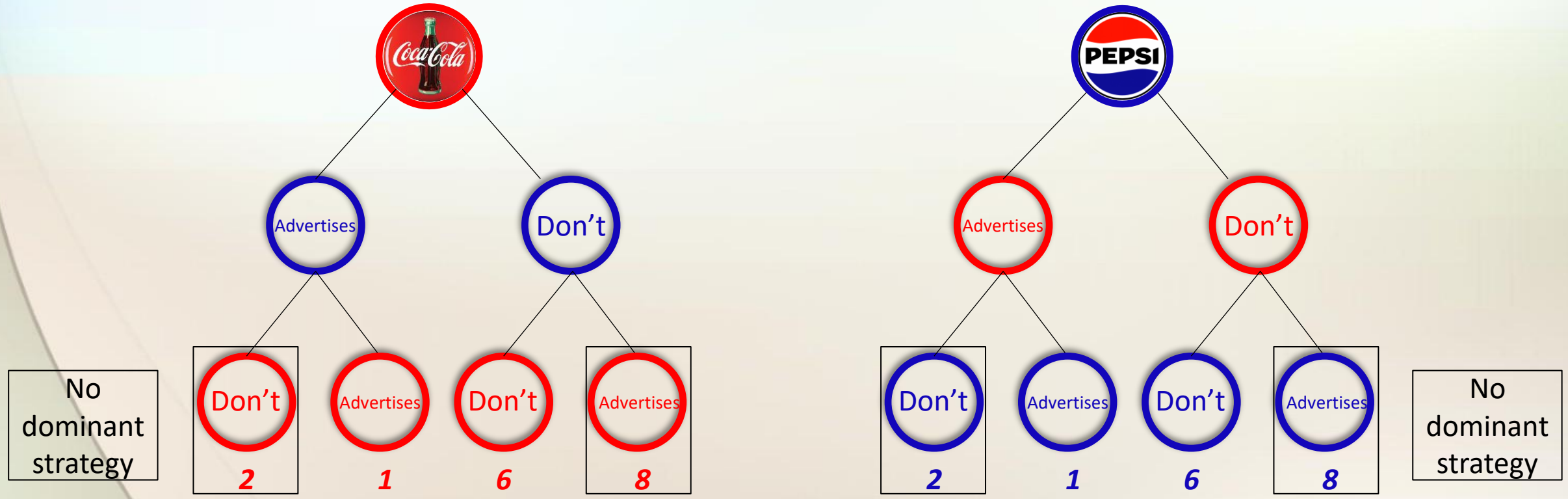
s_{-i}
denotes the strategy profile of all players except player i

$S_{-i} = S_1 \times \cdots \times S_{i-1} \times S_{i+1} \times \cdots \times S_n$
is the set of all possible strategy combinations for players other than i

Sometimes a dominant strategy does not exist!

Example: Dominant Strategy?

(in billions)	*Coke*: Advertise	*Coke*: Don't
***Pepsi*: Advertise**	$(1,1)$	$(8,2)$
***Pepsi*: Don't**	$(2,8)$	$(6,6)$



Iterated Elimination of Dominated Strategies

When analyzing games, we can eliminate strategies that are dominated by others, since rational players would never choose them.

This process can be done iteratively:

1. **Identify and eliminate strictly dominated strategies.** Compare ALL pairwise combinations of strategies for each player. *A strategy is strictly dominated if there exists another strategy that always yields higher payoffs regardless of opponents' choices.*

2. **Recalculate the reduced game**. Remove all dominated strategies from the payoff matrix.

3. **Repeat until no more eliminations are possible.** Continue identifying and eliminating dominated strategies in each successive reduced game.

Expand the tree in its entirety but only consider combinations of strategies.

For a player with n strategies, how many comparisons are required?

$\frac{n(n-1)}{2}$ comparisons for all pairs.

This process helps narrow down the set of reasonable strategy combinations.

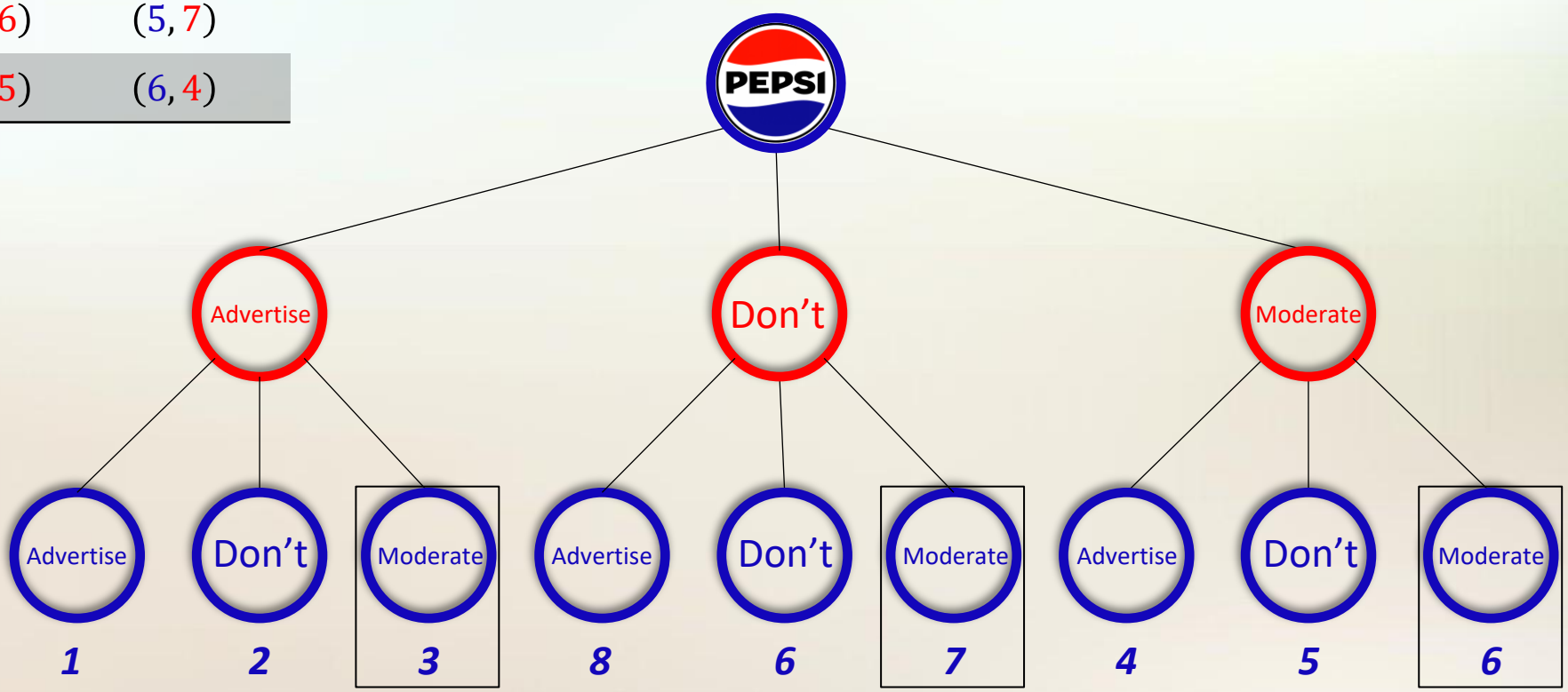
Example: Identifying a Dominated Strategy I

Consider an expanded advertising game with ***Pepsi*** and ***Coca-Cola*** by introducing a selective amount of advertising: 'moderate'.

		Coke		
	(in billions)	**Advertise**	**Don't**	**Moderate**
Pepsi	**Advertise**	(1,1)	(8, 2)	(4, 3)
	Don't	(2, 8)	(6, 6)	(5, 7)
	Moderate	(3, 4)	(7, 5)	(6, 4)

Pepsi

Don't vs. Moderate



Moderate *dominates;*
eliminate ***Don't*** *for* ***Pepsi***

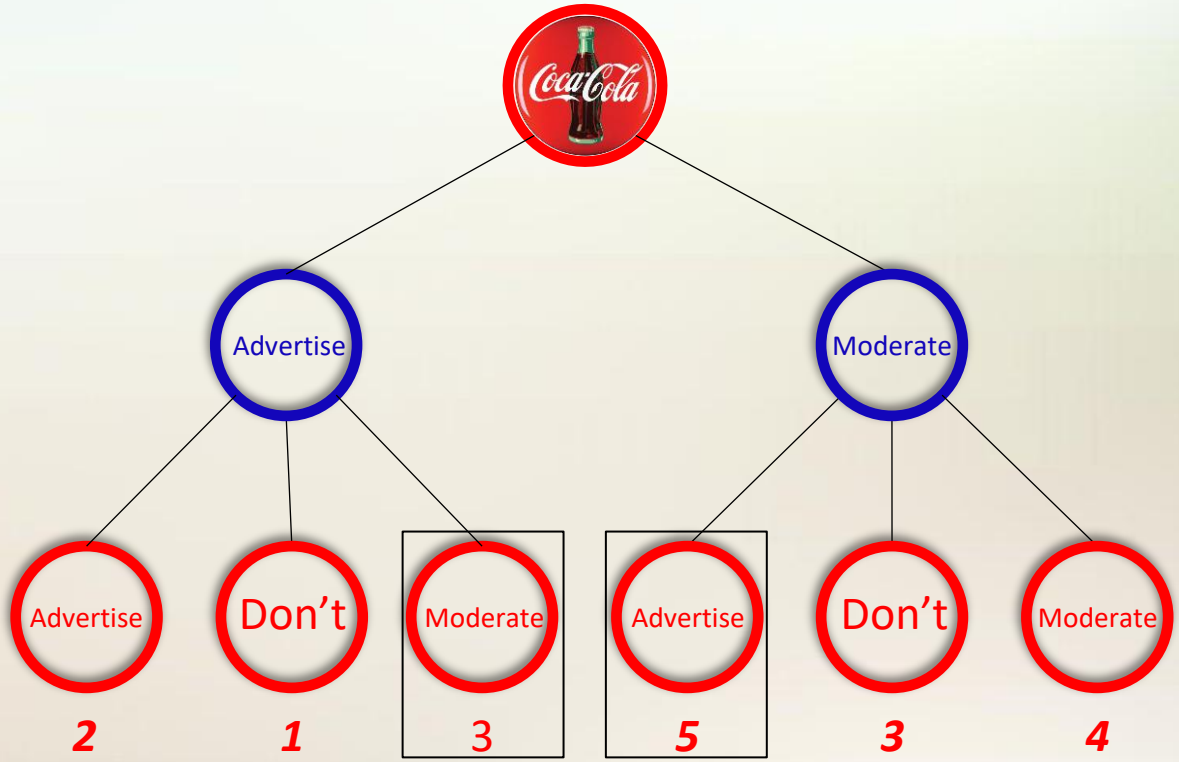
Example: Identifying a Dominated Strategy II

For strict domination, we simplify the game / matrix, by remove **Don't** for ***Pepsi***.

		Coke		
	(in billions)	**Advertise**	**Don't**	**Moderate**
Pepsi	**Advertise**	(2,2)	(8, 1)	(5, 3)
	Moderate	(3, 5)	(6, 3)	(4, 4)

Coke

Advertise vs. Moderate



Neither dominates

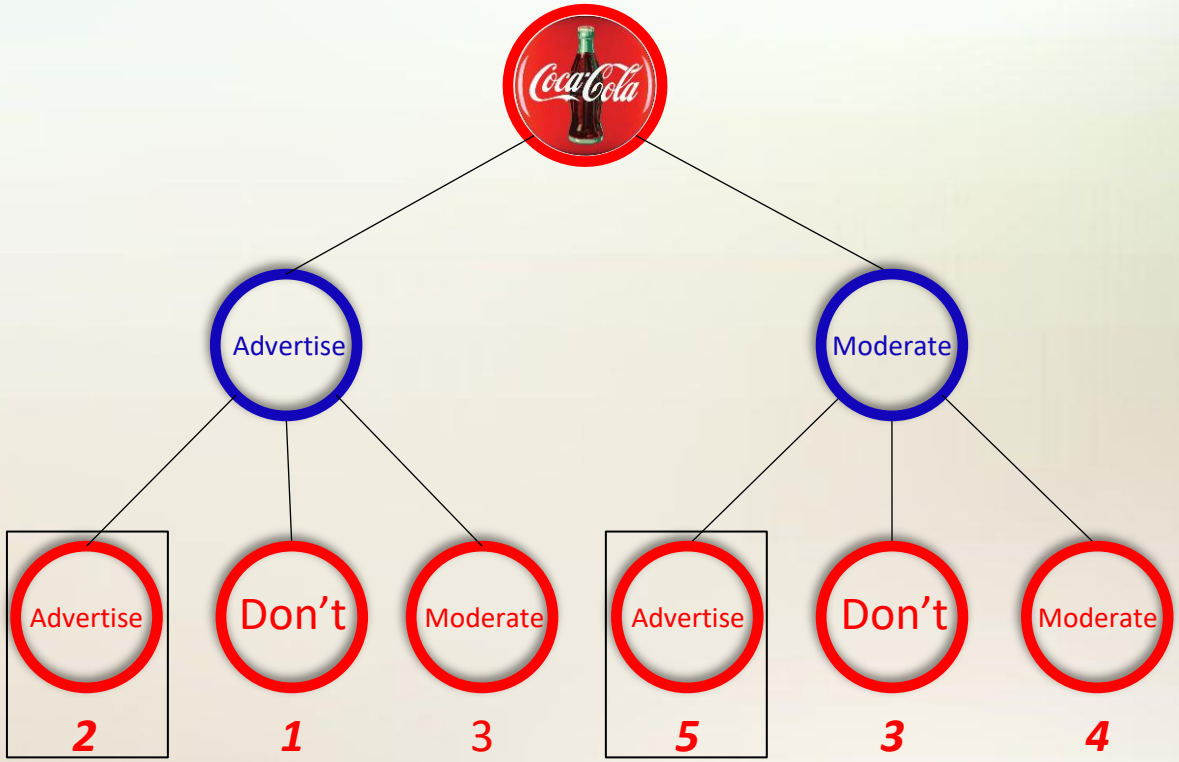
Example: Identifying a Dominated Strategy III

We try another combination.

		Coke		
	(in billions)	**Advertise**	**Don't**	**Moderate**
Pepsi	**Advertise**	(2,2)	(8, 1)	(5, 3)
	Moderate	(3, 5)	(6, 3)	(4, 4)

Coke

Advertise vs. Don't



Advertise *dominates;*
eliminate ***Don't*** *for* ***Coke***

Absolute Best Response = Dominant Strategy

Absolute best response and *dominant strategy* are equivalent terms that describe a strategy that maximizes a player's payoff regardless of what opponents do.

Definition. For player i, strategy s_i is an ***absolute best response*** (***dominant strategy***) if:

$$s_i = \underset{a \in S_i}{\operatorname{argmax}}\, u_i(a, s_{-i})$$

for all possible s_{-i}. (Recall s_{-i} refers to all other players' strategy combinations.)

Dominant Strategy Notation Example

Definition. For player i, strategy s_i is an ***absolute best response*** (***dominant strategy***) if:

$$s_i = \underset{a \in S_i}{\operatorname{argmax}}\, u_i(a, s_{-i})$$

for all possible s_{-i}. (Recall s_{-i} refers to all other players' strategy combinations.)

		Coke	
	(in billions)	**Advertise**	**Moderate**
Pepsi	**Advertise**	(2,2)	(5, 3)
	Moderate	(3, 5)	(4, 4)

Example.

$$\underset{a \in \{Advertise,\, Moderate\}}{\operatorname{argmax}} u_{Pepsi}(a, Advertise) = \underset{a \in \{Advertise,\, Moderate\}}{\operatorname{argmax}} \{2, 3\} = Moderate = s_{Pepsi}$$

$$\neq$$

$$\underset{a \in \{Advertise,\, Moderate\}}{\operatorname{argmax}} u_{Pepsi}(a, Moderate) = \underset{a \in \{Advertise,\, Moderate\}}{\operatorname{argmax}} \{5, 4\} = Advertise = s_{Pepsi}$$

Since $Moderate \neq Advertise$, there is *no single strategy* s_{Pepsi} that satisfies the definition for all values of $= s_{Coke}$. Therefore, Pepsi has no dominant strategy.

Similarly, ***Coke*** does not have a dominant strategy.

Elimination to Equilibrium

We eliminated the bad strategies and we are left with this 2×2 game. Now what? How do we predict what will actually happen?"

		Coke	
	(in billions)	**Advertise**	**Moderate**
Pepsi	**Advertise**	$(2,2)$	$(5,3)$
	Moderate	$(3,5)$	$(4,4)$

We know rational players will not choose dominated strategies, but we still have 4 possible outcomes. Which one will actually occur?

When both players have dominant strategies, we have what is called a ***dominant strategy equilibrium***. This makes the game easy to predict because each player has an obvious best choice regardless of what the opponent does.

In this reduced game, neither company has a dominant strategy. Each company's best choice depends on what the other company does.

We need a different approach.

Nash Equilibrium

Consider each company's perspective and anticipate their strategic moves. What might the other company do?

		Coke	
	(in billions)	Advertise	Moderate
Pepsi	Advertise	$(2,2)$	$(5,3)$
	Moderate	$(3,5)$	$(4,4)$

Check each possible outcome and ask: "Would either company want to change their mind?"

(Advertise, Advertise) $= (2, 2)$

Pepsi: Switching to **Moderate**: $2 \rightarrow 3$ switch

(Advertise, Moderate) $= (5, 3)$

Pepsi: Switching to **Moderate**: $5 \rightarrow 4$

Coke: Switching to **Advertise**: $3 \rightarrow 2$

(Moderate, Advertise) $= (3, 5)$

Pepsi: Switching to **Advertise**: $3 \rightarrow 2$

Coke: Switching to **Moderate**: $5 \rightarrow 4$

(Moderate, Moderate) $= (4, 4)$

Coke: Switching to **Advertise**: $4 \rightarrow 5$ switch

A situation where both companies are content with their choices, given the other's actions, are considered ***stable***.

A mutually reinforcing outcome is known as a ***Nash equilibrium***, and it serves as our solution concept in the absence of dominant strategies.

Nash Equilibria

When we cannot find a single strategy that is always optimal (***absolute best responses***), we instead look for situations where each player's strategy is optimal ***given*** the other player's choice. This shifts us from seeking absolute best responses to finding ***mutual conditional best responses***.

That is exactly what Nash equilibrium captures.

A **Nash equilibrium** occurs when both players are simultaneously playing their best response (argmax) to each other's strategies. It is the natural solution when no player has an absolute best response!

Definition. A strategy profile $\boldsymbol{s}$ is a ***Nash Equilibrium*** if no player has an incentive to unilaterally deviate:

$$\forall i, \forall a \in \boldsymbol{S_i}: u_i(s_i^*, s_{-i}^*) \geq u_i(a, s_{-i}^*).$$

Equivalently, for each player $\boldsymbol{i}$, each player's strategy is a best response to others' equilibrium strategies:

$$s_i^* \in \boldsymbol{BR_i}(s_{-i}^*) = \underset{a \in S_i}{\operatorname{argmax}}\, u_i(a, s_{-i}^*).$$

- $\boldsymbol{s}$: any arbitrary strategy profile from the strategy spaces
- $\boldsymbol{s^*}$: equilibrium strategy profile (the specific combination we are analyzing)
- $\boldsymbol{s_i^*}$: player $\boldsymbol{i}$'s equilibrium strategy (what they actually play in equilibrium)
- $\boldsymbol{s_{-i}^*}$: other players' equilibrium strategies (fixed at equilibrium values)
- $\boldsymbol{a}$: any alternative strategy player $\boldsymbol{i}$ might deviate to
- $\boldsymbol{BR_i}$: Best Response function for player $\boldsymbol{i}$
- $\boldsymbol{S_i}$: Strategy set for player $\boldsymbol{i}$ (all available strategies)

Nash Equilibria vs. Dominant Strategy Equilibrium:

Nash Equilibrium

- Player $\boldsymbol{i}$'s strategy is optimal given the specific strategies others choose in equilibrium
- ***Conditional optimality***: "My strategy is optimal given what you are actually doing."

$$s_i^* = \boldsymbol{BR_i}(s_{-i}^*)$$

Dominant Strategy Equilibrium

- Player $\boldsymbol{i}$'s strategy is optimal against every possible strategy combination by others
- ***Unconditional optimality***: "My strategy is optimal no matter what you do."

$$s_i^* = \boldsymbol{BR_i}(s_{-i}), \forall s_{-i}$$

DSE is more restrictive: it requires optimality against ALL possible opponent behaviors.

Every DSE is automatically a Nash equilibrium, but Nash equilibria can exist without dominant strategies (as in our ***Coke***-***Pepsi*** game).

Computing Pure Nash Equilibria

A ***pure Nash equilibrium*** is an equilibrium where every player uses a pure strategy: each player picks exactly one action and sticks to it deterministically. A mixed Nash equilibrium involves at least one player randomizing between strategies.

A ***mixed strategy*** involves randomizing over multiple actions according to some probability distribution. For example, "advertise 60% of the time, do not advertise 40% of the time."

Computing (pure) Nash Equilibria:

1

Find (and highlight) their best response to each of the opponent's strategies.

- As player 1, identify the highest payoff in each column.
- As player 2, identify the highest payoff in each row.

2

A ***pure Nash equilibrium*** exists where both players are simultaneously playing best responses to each other: where both players have highlighted.

Example: Pure Nash Equilibria

1. Identify best ***responses*** (P1) column- and (P2) row-wise.
2. Any overlap?

Reconsider the ***Pepsi*** vs ***Coke*** advertising game we saw earlier. Each had three strategies.

		Coke		
	(in billions)	Advertise	Don't	Moderate
Pepsi	Advertise	(1,1)	(8, 2)	(4,3)
	Don't	(2,8)	(6, 6)	(5, 7)
	Moderate	(3, 4)	(7,5)	(6, 4)

There is no pure Nash Equilibrium here (no overlapping marks).

	Stay Silent	Betray
Stay Silent	$(-1,-1)$	$(-3,0)$
Betray	$(0\ -3)$	$(-2,-2)$

Pure Nash Equilibrium: betraying is a dominant strategy.

Nash equilibrium represents stability (no incentive to change), not optimality.

When Nash Equilibrium Does Not Exist

So we have pure strategy: each player picks a deterministic strategy.

Many zero-sum games lack pure Nash equilibria since one player's gain is another player's loss.

This creates inherent conflict that prevents stable, pure strategies.

Some competitive situations inherently require unpredictability.

What happens when no pure strategy Nash equilibrium exists?

	Rock	**Paper**	**Scissors**
Rock	$(0,0)$	$(-1,1)$	$(1,-1)$
Paper	$(1,-1)$	$(0,0)$	$(-1,1)$
Scissors	$(-1,1)$	$(1,-1)$	$(0,0)$

Rock-Paper-Scissors has no pure Nash Equilibrium.

Nash's Key Insight

Even when pure Nash equilibria do not exist, we can find equilibria using ***mixed strategies***.

Mixed Strategies

	Rock	Paper	Scissors
Rock	$(0,0)$	$(-1,1)$	$(1,-1)$
Paper	$(1,-1)$	$(0,0)$	$(-1,1)$
Scissors	$(-1,1)$	$(1,-1)$	$(0,0)$

A ***mixed strategy*** refers to assigning probabilities to each pure strategy in a player's strategy set.

Example

Pure strategy:

Choose one action with 100% probability

"Always play Heads"

Mixed strategy:

Randomize over actions according to probabilities

"Play Heads 60% of the time, Tails 40% of the time"

Recall, we are talking about simultaneous move games, so:

- randomization is private and
- opponents will not know the outcome until after you play.

Mixed Strategies II

Definition. A ***mixed strategy*** for player $\boldsymbol{i}$ is a probability distribution σ_i over player $\boldsymbol{i}$'s pure strategy set A_i, where:

$$\sigma_i : \boldsymbol{S_i} \to [0,1] \text{ such that } \sum_{a_i \in \boldsymbol{S_i}} \sigma_i(a_i) = 1$$

where $\sigma_i(a_i)$ represents the probability that player $\boldsymbol{i}$ chooses pure strategy a_i.

Example.

With Rock, Paper, Scissors, suppose Player 1 has a uniform strategy profile and player 2 does not:

$\sigma_1 : \boldsymbol{S_1} \to [0,1]$ where $\boldsymbol{S_1} = \{Rock, Paper, Scissors\}$ with

$$\sigma_1(Rock) = \sigma_1(Paper) = \sigma_1(Scissors) = \frac{1}{3}$$

$\sigma_2 : \boldsymbol{S_2} \to [0,1]$ where $\boldsymbol{S_2} = \{Rock, Paper, Scissors\}$ with

$$\sigma_2(Rock) = \frac{5}{10} \quad \sigma_2(Paper) = \frac{3}{10} \quad \sigma_2(Scissors) = \frac{2}{10}$$

Expected Utility Under Mixed Strategies

Definition [Expected utility under mixed strategies]. Given a mixed strategy profile (one for each of the n players) $\boldsymbol{\sigma} = (\sigma_1, \sigma_2, \dots \sigma_n)$, player $\boldsymbol{i}$'s expected utility is:

$$u_i(\sigma_i, \sigma_{-i}) = \sum_{a_i \in S_i} \sum_{a_{-i} \in S_{-i}} \sigma_i(a_i) \cdot \sigma_{-i}\,(a_{-i}) \cdot u_i(a_i, a_{-i}) \quad \text{where:}$$

σ_{-i} denotes the mixed strategy profile of all players except $\boldsymbol{i}$

$\boldsymbol{S_{-i}}$ denotes the Cartesian product of all other players' strategy sets

$\sigma_{-i}(a_{-i}) = \prod_{j \neq i} \sigma_j(a_j)$ is the probability that other players choose action profile a_{-i}

$u_i(a_i, a_{-i})$ is player $\boldsymbol{i}$'s payoff from the pure strategy profile (a_i, a_{-i})

Alternatively,

$$u_i(\sigma_i, \sigma_{-i}) = E[a_i \sim \sigma_i][u_i(a_i, a_{-i})]$$

where the expectation is taken over the random draws from each player's mixed strategy.

$\sim$ means "drawn from": $a_i \sim \sigma_i$ means "a_i is drawn from distribution σ_i".

Example: Expected Utility Under Mixed Strategies

$$u_i(\sigma_i, \sigma_{-i}) = \sum_{a_i \in S_i} \sum_{a_{-i} \in S_{-i}} \sigma_i(a_i) \cdot \sigma_{-i}(a_{-i}) \cdot u_i(a_i, a_{-i})$$

$\sigma_2(Rock) = \frac{5}{10}$ $\sigma_2(Paper) = \frac{3}{10}$ $\sigma_2(Scissors) = \frac{2}{10}$

Example continued.

Player 1's expected utility:

$$\begin{aligned} u_1(\sigma_1, \sigma_{-1}) = {} & \sigma_1(R) \cdot \sigma_{-1}(R) \cdot u_1(R, R) + \sigma_1(R) \cdot \sigma_{-1}(P) \cdot u_1(R, P) + \sigma_1(R) \cdot \sigma_{-1}(S) \cdot u_1(R, S) + \\ & \sigma_1(P) \cdot \sigma_{-1}(R) \cdot u_1(P, R) + \sigma_1(P) \cdot \sigma_{-1}(P) \cdot u_1(P, P) + \sigma_1(P) \cdot \sigma_{-1}(S) \cdot u_1(P, S) + \\ & \sigma_1(S) \cdot \sigma_{-1}(R) \cdot u_1(S, R) + \sigma_1(S) \cdot \sigma_{-1}(P) \cdot u_1(S, P) + \sigma_1(P) \cdot \sigma_{-1}(P) \cdot u_1(P, P) \\ = {} & \left(\frac{1}{3}\right)\left(\frac{5}{10}\right)(0) + \left(\frac{1}{3}\right)\left(\frac{3}{10}\right)(-1) + \left(\frac{1}{3}\right)\left(\frac{2}{10}\right)(1) + \\ & \left(\frac{1}{3}\right)\left(\frac{5}{10}\right)(1) + \left(\frac{1}{3}\right)\left(\frac{3}{10}\right)(0) + \left(\frac{1}{3}\right)\left(\frac{2}{10}\right)(-1) + \\ & \left(\frac{1}{3}\right)\left(\frac{5}{10}\right)(-1) + \left(\frac{1}{3}\right)\left(\frac{3}{10}\right)(1) + \left(\frac{1}{3}\right)\left(\frac{2}{10}\right)(0) \\ = {} & 0 \end{aligned}$$

Similarly, Player 2's expected utility:

$$u_2(\sigma_2, \sigma_{-2}) = 0$$

Or, as a zero-sum game:

$$u_2(\sigma_2, \sigma_{-2}) = -u_1(\sigma_1, \sigma_{-1}) = 0$$

Both players have expected utility 0, even with Player 2's non-uniform strategy.

This means we have a fair game: no player has a net advantage.

Finding Mixed NE in 2-Player, 2-Action Zero-Sum Game

Even / Odd 2-finger Morra is a game of chance and strategy, typically played by two people.

Objective: To correctly guess whether the **sum of fingers** shown by both players is **even** or **odd**.

How to Play:

1. **Choose Roles**: One player is "Even," the other is "Odd."
2. **Simultaneous Play**:
   - Each player throws out *one or two fingers*.
   - The total number of fingers is summed.
3. **Determine Winner**: If the total is even, the "Even" player wins; otherwise, the "Odd" player wins.

Assume the payoff matrix:

		Odd	
		1 finger	**2 fingers**
Even	**1 finger**	$(2, -2)$	$(-3, 3)$
	2 fingers	$(-3, 3)$	$(4, -4)$

No pure NE.

Even / Odd Morra

Applying Nash equilibrium to Morra:
Player 1 = Even, Player 2 = Odd.

Let σ_{even} be the mixed strategy:

$$\left(\sigma_{even}(1), \sigma_{even}(2)\right) = (p, 1-p)$$

p is the probability that the even player selects f_1.

Let σ_{odd} be the mixed strategy:

$$\left(\sigma_{odd}(1), \sigma_{odd}(2)\right) = (q, 1-q)$$

q is the probability that the odd player selects "1 finger": f_1.

By definition of *best response*, p is the conditional best response to q. That is,

$$u_1(p, q) \geq u_1(p', q)$$

for all p' in the set of all possible mixed strategies for Player 1.

By the definition of expected utility for mixed utility games (for Player 1):

$$u_{even}(p, q) = p \cdot q \cdot u_{even}(f_1, f_1) + p \cdot (1-q) \cdot u_{even}(f_1, f_2) + (1-p) \cdot q \cdot u_{even}(f_2, f_1) + (1-p) \cdot (1-q) \cdot u_{even}(f_2, f_2)$$

$$= p \cdot [q \cdot u_{even}(f_1, f_1) + (1-q) \cdot u_{even}(f_1, f_2)] + (1-p) \cdot [q \cdot u_{even}(f_2, f_1) + (1-q) \cdot u_{even}(f_2, f_2)]$$

$$= p \cdot u_{even}(f_1, q) + (1-p) \cdot u_{even}(f_2, q)$$

Player 1 plays f_1 against mixed strategy q: $u_{even}(f_1, q)$

Player 1 plays f_2 against mixed strategy q: $u_{even}(f_2, q)$

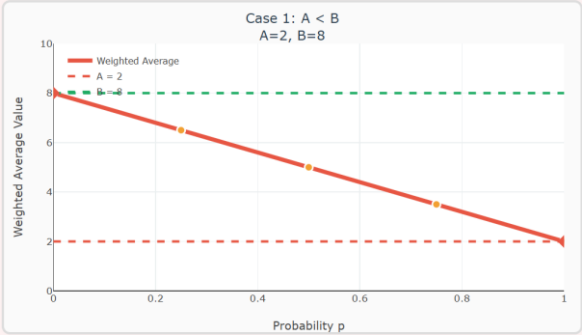
Even / Odd Morra II

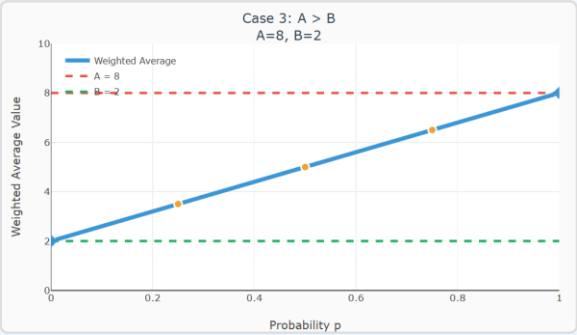
1. We have p is the optimal strategy for the even player: $u_{even}(p,q) \geq u_{even}(p',q), \forall p'$

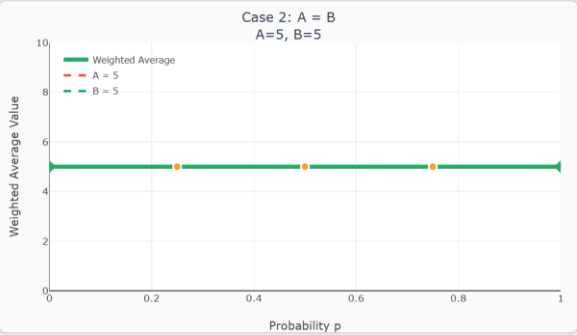
2. With expected utility for the even player, we have a *weighted average* of two utility values: $u_{even}(p,q) = p \cdot u_{even}(f_1,q) + (1-p) \cdot u_{even}(f_2,q)$

Since p is optimal, it must do at least as well as the pure strategies f_1 and f_2:

$$u_{even}(p,q) \geq u_{even}(f_1,q) \qquad u_{even}(p,q) \geq u_{even}(f_2,q)$$







A weighted average can only be greater than or equal to both of its components *if those components are equal*.

Therefore, suppose: $u_{even}(f_1,q) = u_{even}(f_2,q) = k$ where k is a payoff value.

Then, $u_{even}(p,q) = p \cdot k + (1-p) \cdot k = k$.

Thus, $u_{even}(p,q) = u_{even}(f_1,q) = u_{even}(f_2,q)$.

We conclude: the even player gets the same expected payoff whether they play 1 finger, 2 fingers, or mix between them.

Even / Odd Morra III

Even \ Odd	1 finger	2 fingers
1 finger	$(2,-2)$	$(-3,3)$
2 fingers	$(-3,3)$	$(4,-4)$

Recall, the expected utility under mixed strategies:

$$u_i(\sigma_i,\sigma_{-i}) = \sum_{a_i \in S_i} \sum_{a_{-i} \in S_{-i}} \sigma_i(a_i) \cdot \sigma_{-i}(a_{-i}) \cdot u_i(a_i, a_{-i})$$

Also recall we showed,

$$u_{even}(f_1,q) = u_{even}(f_2,q).$$

Combining, we have: $u_{even}(f_1,q) = 2 \cdot q + (-3)(1-q) = u_{even}(f_2,q) = (-3)q + 4(1-q)$

Solving, $2 \cdot q + (-3)(1-q) = (-3)q + 4(1-q) \Leftrightarrow q = \frac{7}{12}$

Similarly, $u_{odd}(p,f_1) = u_{odd}(p,f_2) \Leftrightarrow p = \frac{7}{12}$

Thus, for $q = \frac{7}{12}$, $u_{even}\left(p, \frac{7}{12}\right) = 2 \cdot \frac{7}{12} + (-3)\left(1 - \frac{7}{12}\right) = -\frac{1}{12}$

Similarly, for $p = \frac{7}{12}$, $u_{odd}(p,q) = -2 \cdot p + 3 \cdot (1-p) = 3 \cdot p + (-4) \cdot (1-p) = \frac{1}{12}$

Observe, mixed equilibrium probabilities depend only on opponent's payoffs, not your own!

Even though both players play optimally, the game's structure gives the odd player an inherent advantage of $\frac{1}{12}$ per round on average.

The Indifference Principle

If a strategy yielded higher expected payoff → a player would use only that strategy

For ***optimal*** mixing → all pure strategies in the mix must yield *equal* expected payoffs

Counterintuitive
You choose probabilities to make your opponent's strategies equally good

Not
Probabilities that directly maximize your own expected payoff

Process / Intuition with 2-Finger Morra

When ***odd*** uses a mixed strategy ($\frac{7}{12}$ for 1-finger), ***even***'s pure actions both yield $-\frac{1}{12}$ expected payoff.

Since both pure actions are equally good, ***even*** is indifferent and could mix in any proportion

But for equilibrium, ***even*** must choose the specific mix ($\frac{7}{12}$ for 1-finger) that makes ***odd*** indifferent.

When ***even*** uses this mix, ***odd***'s pure actions both yield $+\frac{1}{12}$ expected payoff.

Recall, a ***mixed strategy*** refers to a probabilistic combination of pure actions

In mixed strategy equilibrium, deviating from your optimal mixing probabilities does not directly hurt your payoff, but it breaks your opponent's indifference and destroys the equilibrium. (Thus your opponent would move to a pure strategy.)

This is a ***mutual dependency*** since each player's mixing creates the other's willingness to mix.

Defining The Indifference Principle

We saw an example of the process in the previous slide.

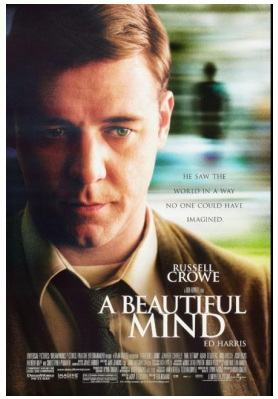
Each player's mixing probabilities are determined by making their opponent's strategies equally attractive

Player A gets equal payoffs (indifference) → determines Player B's optimal mixing probabilities
Player B gets equal payoffs (indifference) → determines Player A's optimal mixing probabilities

Definition. The indifference principle is the *method* for finding mixed strategy equilibria (where no player wants to change their strategy). In these equilibria, each player gets equal expected payoffs from all pure actions in their optimal mix.

In other words, players do not maximize their own payoff directly; instead, they choose probabilities that make their opponent indifferent.

The Breakthrough of John Nash



John Nash's Breakthrough (1951)

- Ph.D. from Princeton at age 22, completed in two years
- His 1950 PNAS paper: just one page, revolutionized game theory
- Initially dismissed by von Neumann; now one of the most widely used tools in economics

The Revolution

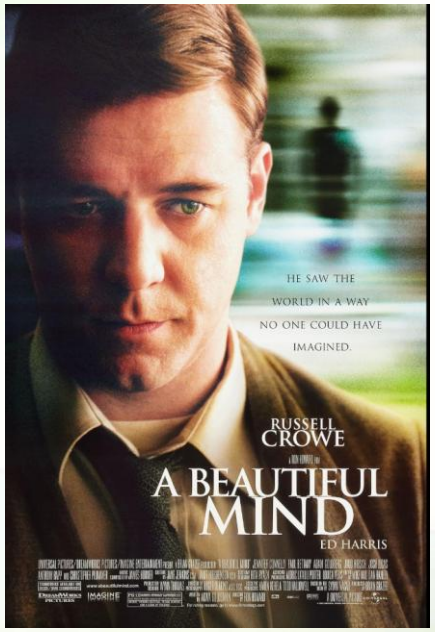
Von Neumann & Morgenstern	Nash
Zero-sum games only	All finite games
One player's gain = other's loss	Players can both win or both lose
Limited real-world application	Applicable to competition and cooperation

Mathematical innovation. Nash used **Kakutani's fixed-point theorem** to prove existence:

- Maps points to *sets* of points (not single points)
- Essential because players often have multiple equally good strategies
- Guarantees at least one stable equilibrium exists in every finite game

Nash's Existence Theorem

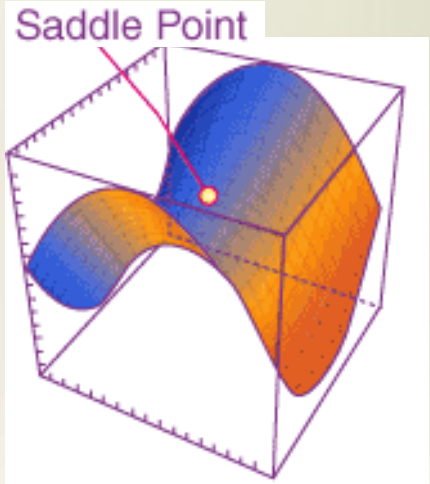
Theorem [Nash Equilibrium Existence]. Every game with a finite number of players and a finite number of strategies per player has at least one Nash equilibrium in pure or mixed strategies.



Existence guarantees:

- At least one equilibrium exists
- A stable solution where no player wants to deviate

For a 2-player zero-sum game, the equilibrium is where $minimax = maximin$: Player A can't climb higher, Player B can't push lower. Neither can improve unilaterally.



Existence does NOT guarantee:

- Uniqueness (multiple equilibria possible)
- Pareto efficiency (both players could be better off, but aren't)
- Finding equilibria is computationally challenging for general games

Question

Which of the following are ***true***?

i. Nash equilibria require each player to know other players' possible strategies ✓
ii. Nash equilibria require rational play ✓

1. Neither
2. (i) but not (ii)
3. (ii) but not (i)
4. ✓ Both

Nash equilibrium assumes players know all available strategies (complete information)

Nash equilibrium assumes rational, utility-maximizing players

Which of the following are ***false***?

i. Rock-paper-scissors has a dominant pure strategy ✗
ii. There is a Nash equilibrium for rock-paper-scissors ✓

1. Neither
2. ✓ (i) but not (ii)
3. (ii) but not (i)
4. Both

RPS has no dominant strategy: each option loses to one and beats another

RPS has a mixed Nash equilibrium where each player plays (1/3, 1/3, 1/3)

Connection to AI Search

When the environment becomes an intelligent adversary with its own objectives, optimization becomes strategic reasoning:

Search → Game Theory.

Single-Agent Search	Game Theory
A* algorithm	Minimax algorithm
Optimal path	Optimal strategy
Heuristic functions	Evaluation functions
State space	Game tree
Goal state	Payoff / utility

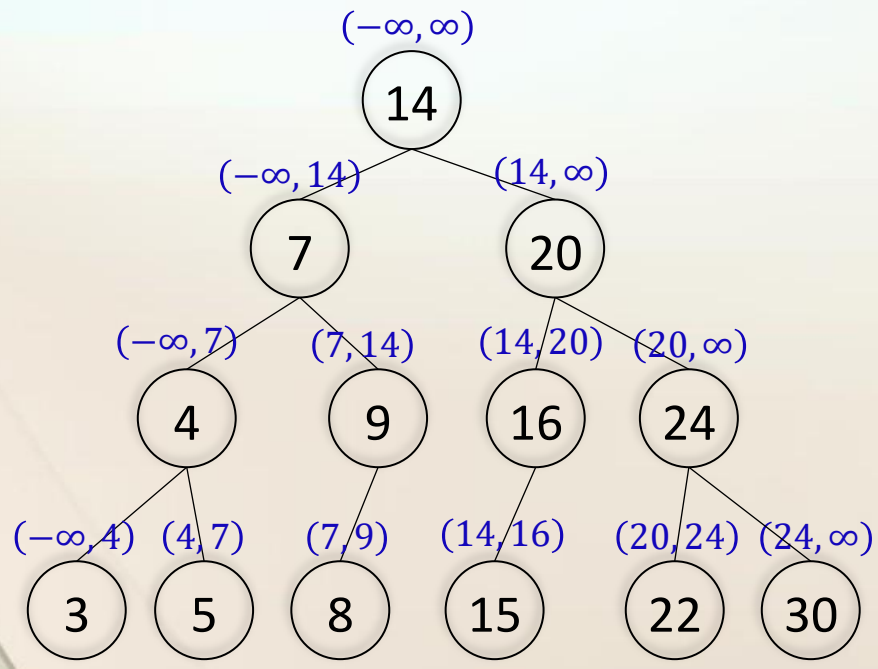
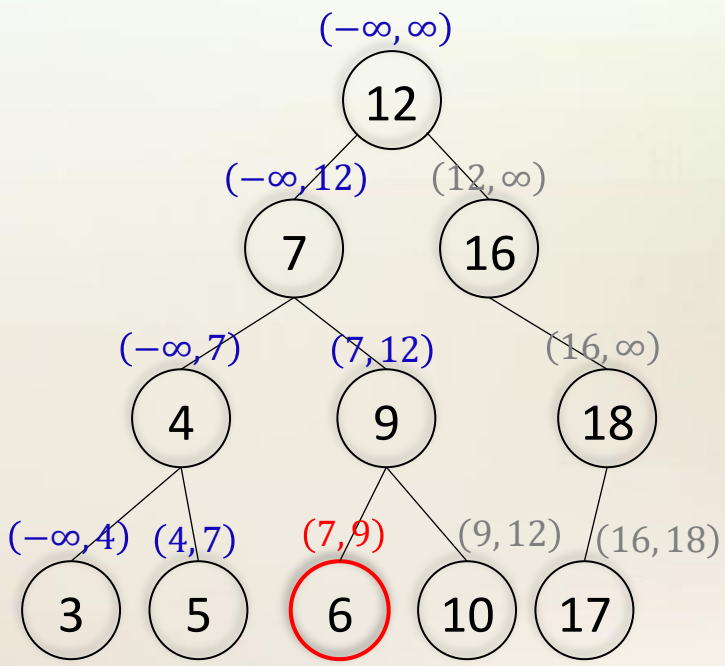
The Fundamental Shift

Search: Find the best sequence of actions to reach a goal
Games: Find the best strategy assuming opponents play optimally against you

A Blast From the Past: Binary Search Tree

Give linear time algorithm to determine if a binary search tree maintains the symmetric ordering property (i.e., left subtree values $<$ root $<$ right subtree values).

Solution.

Consider a "bounds propagation" approach in which each node must be between: $(left, right)$.
As we traverse:

- **Go left**: update the $right$ bound with `node.val` (nodes in the left subtree must be *less* than or equal `node.val`)
- **Go right**: update the $left$ bound with `node.val` (nodes in the right subtree must be *greater* than `node.val`)